

1. Uvod

Razumevanje izvršavanja programa na nivou asemblerskog jezika podrazumeva praćenje veze između instrukcija, sadržaja registara, memorije i redosleda izvršavanja. Konačan rezultat programa ne pokazuje sva međustanja kroz koja je procesor prošao da bi do njega došao. Zbog toga je za analizu rada programa značajno posmatrati ne samo izlazne vrednosti, već i promene koje nastaju izvršavanjem pojedinačnih instrukcija.

Motivacija za ovaj rad jeste izrada okruženja u kome se kratak asemblerski program može napisati, izvršiti i analizirati uz neposredan uvid u stanje simuliranog procesora. Na primeru petlje koja sabira niz brojeva, korisnik treba da može da prati promenu zbira, vrednost brojača i trenutak u kome uslov grananja prestaje da bude ispunjen. Takav način rada namenjen je povezivanju tekstualnog zapisa programa sa njegovim izvršavanjem, bez potrebe da se za svaki korak ručno vodi evidencija svih promena.

Predmet rada vezan je za RISC-V, porodicu arhitektura skupa instrukcija, odnosno pravila koja određuju instrukcije i stanje procesora vidljivo programu. Specifikacija RISC-V razdvaja osnovni celobrojni skup instrukcija od dodatnih proširenja, pri čemu RV32I označava osnovnu varijantu sa 32-bitnim celobrojnim registrima [1]. U ovom radu razmatra se odabrani podskup instrukcija te varijante, dovoljan za primere aritmetičkih i logičkih operacija, pristupa memoriji i promene toka izvršavanja.

Predmet praktičnog dela rada jeste razvoj veb aplikacije koja objedinjuje editor asemblerskog koda, parser, assembler i simulator izvršavanja. Korisnik unosi program u tekstualni editor, nakon čega aplikacija proverava njegov zapis i prevodi ga u internu reprezentaciju pogodnu za izvršavanje. Ako je program ispravan, prelazi se u režim u kome se može upravljati izvršavanjem i pratiti stanje registara i memorije.

Osnovni cilj je realizacija preglednog alata koji omogućava izvršavanje jedne instrukcije, nastavak izvršavanja do tačke prekida i izvršavanje do završetka programa. Dodatni ciljevi obuhvataju prikaz promena stanja posle korisničke akcije, prijavljivanje grešaka sa vezom ka izvornom kodu i vraćanje simuliranog procesora u početno stanje. Pri tome je potrebno sačuvati vezu između instrukcije koju simulator izvršava i odgovarajuće linije koju korisnik vidi u editoru, kako bi praćenje programa ostalo razumljivo i nakon prevođenja pseudo-instrukcija.

Obim rešenja ograničen je na 15 instrukcija i 5 pseudo-instrukcija. Podržane instrukcije obuhvataju sabiranje i oduzimanje, bitske logičke operacije i pomeranja, učitavanje i upis memorijske reči, uslovno grananje i skokove. Model stanja sadrži 32 registra širine 32 bita, programski brojač i memoriju podataka veličine 4096 bajtova, dok se pseudo-instrukcije u fazi asembliranja prevode u podržane prave instrukcije.

Simulator predstavlja model izvršavanja na nivou instrukcija, a ne model unutrašnje hardverske organizacije procesora. Asemlirani program čuva se kao niz strukturiranih instrukcija, odvojeno od memorije podataka, bez generisanja binarnog mašinskog koda. Ne modeluju se protočna obrada, keš memorija niti trajanje instrukcija u taktovima, pa prikazani rezultati opisuju promene programskog stanja, a ne performanse stvarnog procesora.

Postupak izrade zasnovan je na razdvajanju obrade izvornog koda, izvršavanja i korisničkog interfejsa. Parser izdvaja strukturu linija i operanada, a assembler proverava da li zapis odgovara podržanim instrukcijama i u dva prolaza razrešava labele i priprema program. Model procesora potom izvršava pripremljene instrukcije i menja registre, memoriju i programski brojač, nezavisno od načina na koji se te promene prikazuju korisniku.

Za implementaciju je korišćen TypeScript, dok je korisnički interfejs realizovan veb tehnologijama HTML i CSS i neposrednim radom sa objektnim modelom dokumenta, odnosno DOM-om. Vite se koristi za razvojno okruženje i pripremu aplikacije za isporuku, a Vitest za automatizovane testove. Logika parsera, assemblera i modela procesora ne zavisi od elemenata veb stranice, što omogućava njenu proveru odvojeno od prikaza.

Interakcija sa programom organizovana je kroz režim pisanja i režim izvršavanja. U režimu izvršavanja komanda Step izvršava jednu instrukciju, dok Run nastavlja rad do završetka, tačke prekida ili greške. Prikaz izdvaja sledeću instrukciju, registre u koje je upisivano i memorijske reči koje su menjane, a Reset omogućava ponovno izvršavanje učitanoog programa od početnog stanja.

Prijavljivanje grešaka obuhvata i obradu izvornog teksta i samo izvršavanje. Neispravan zapis instrukcije ili nedefinisana labela sprečavaju pokretanje programa, dok nedozvoljen pristup memoriji ili neispravno odredište skoka prekidaju izvršavanje. Uvedeno je i ograničenje broja instrukcija u jednom pozivu komande Run, čime se ograničava trajanje izvršavanja programa koji ne dolazi do završetka ili tačke prekida.

Provera rešenja zasniva se na automatizovanim testovima komponenti i ručnoj proveru korisničkog interfejsa. Testovi obuhvataju ispravne programe i granične slučajeve, kao što su prekoračenje dozvoljenog opsega konstante, 32-bitno preliivanje, neporavnat pristup memoriji i ponašanje grananja i skokova. Rezultati ovih provera pokazuju kako se implementirane funkcije ponašaju u ispitanim slučajevima. Oni ne predstavljaju formalni dokaz ispravnosti, a na osnovu njih se ne može zaključiti koliko alat pomaže korisnicima u učenju.

Rezultat praktičnog dela jeste integrisana aplikacija u kojoj se program može pratiti od unosa izvornog teksta do promene stanja simuliranog procesora. Doprinos rada je u projektovanju, povezivanju i proveru komponenti koje ostvaruju zadate funkcionalnosti u okviru jasno ograničenog skupa instrukcija. Namenjena je demonstraciji i analizi kraćih asemblerskih programa, dok podrška celokupnom skupu instrukcija i složenijem razvojnom okruženju ostaje izvan obima ovog rešenja.

Rad je organizovan u osam poglavlja. Posle uvoda, drugo poglavlje daje pregled postojećih rešenja i kriterijuma za njihovo poređenje, dok treće objašnjava teorijske osnove potrebne za razumevanje simulatora. Četvrto poglavlje predstavlja korišćene tehnologije, a peto arhitekturu i implementacione detalje sistema.

U šestom poglavlju prikazane su funkcionalnosti aplikacije kroz primer korišćenja i praćenja izvršavanja programa. Sedmo poglavlje opisuje postupak testiranja, rezultate i ograničenja sprovedene provere. Osmo poglavlje donosi zaključak o ostvarenim ciljevima i razmatra moguća unapređenja rešenja.

2. Pregled postojećih rešenja

Za poređenje sa simulatorom razvijenim u ovom radu odabrani su alati RARS i Venus. Oba rešenja namenjena su radu sa RISC-V asemblerskim programima i omogućavaju praćenje njihovog izvršavanja [2] [4]. Pregled je usmeren na mogućnosti značajne za ovaj rad, a ne na iscrpan popis svih funkcija ili rangiranje alata.

2.1. Kriterijumi izbora i poređenja

RARS i Venus izabrani su zato što objedinjuju obradu asemblerskog programa i simulaciju, umesto da zahtevaju fizički procesor za izvršavanje. Njihova dokumentovana podrška za izvršavanje korak po korak i tačke prekida odgovara osnovnim zahtevima razvijenog rešenja [2-4]. Istovremeno predstavljaju dva različita načina korišćenja: RARS se distribuira kao Java aplikacija, dok je Venus dostupan kroz veb pregledač.

Porede se način pokretanja alata, podržani skup instrukcija, kontrole izvršavanja, prikaz registara i memorije, kao i mogućnost uporednog prikaza izvornog i mašinskog koda. Razmatraju se i dodatne mogućnosti, poput vraćanja prethodnog koraka i poziva simuliranog okruženja. Ovi kriterijumi omogućavaju da se funkcionalnosti alata povežu sa konkretnim zadacima: praćenjem petlje, analizom skoka, posmatranjem upisa u memoriju i razumevanjem prevođenja instrukcija.

Pregled je zasnovan na dokumentaciji projekata i dostupnom interfejsu Venus-a. Nisu sprovedena uporedna merenja brzine, sistematska provera usaglašenosti sa specifikacijom niti istraživanje upotrebljivosti sa korisnicima. Na osnovu ovog pregleda mogu se porediti opisane mogućnosti alata, ali se ne može oceniti njihova brzina, potvrditi potpuna ispravnost niti utvrditi koliko pomažu u učenju.

2.2. RARS

RARS, odnosno RISC-V Assembler and Runtime Simulator, predstavlja okruženje za asembliranje i simulaciju RISC-V programa. Distribuira se u obliku izvršne JAR datoteke, za čije pokretanje je prema uputstvu potrebna Java 8 ili novija verzija. Projekat navodi podršku za 32-bitnu i 64-bitnu varijantu arhitekture, više proširenja skupa instrukcija, pozive okruženja i asembliranje programa iz više datoteka [2].

Interaktivno izvršavanje odvija se u prikazu Execute. Komanda Step omogućava praćenje pojedinačnih instrukcija, a Go neprekidno izvršavanje ili nastavak nakon zaustavljanja; tačke prekida postavljaju se uz prikazane instrukcije. Dokumentacija opisuje i podešavanje brzine simulacije, pauziranje, ponovno pokretanje iz početnog stanja i prikaz registara i memorijskih vrednosti u decimalnom ili heksadecimalnom zapisu [3].

Posebno je značajna mogućnost poređenja pseudo-instrukcija sa osnovnim instrukcijama i mašinskim kodom [2]. Ona omogućava razmatranje veze između pogodnijeg zapisa programa i njegovog stvarnog kodiranja, što u razvijenom simulatoru nije predmet prikaza. RARS podržava i korake unazad, ali dokumentacija naglašava da poništavanje promena registara i memorije ne obuhvata konzolni i datotečki ulaz i izlaz [3].

U odnosu na zadati obim ovog rada, RARS obezbeđuje šire razvojno okruženje. To je relevantno za programe koji koriste dodatne instrukcije, više izvornih datoteka ili pozive okruženja, ali nije zahtev za realizaciju ovde izabranih primera. Potreba za Java okruženjem predstavlja razliku u načinu pokretanja, a ne sama po sebi nedostatak funkcionalnosti.

2.3. *Venus*

Venus je simulator RISC-V instrukcija namenjen obrazovanju. U ovom pregledu razmatraju se projekat kvakil/venus i veb aplikacija na koju taj projekat upućuje, bez pripisivanja mogućnosti drugih izvedenih verzija. Prema dokumentaciji, podržan je RV32IM, odnosno osnovni 32-bitni celobrojni skup instrukcija sa proširenjem za množenje i deljenje [1] [4].

Na kartici Editor korisnik piše program, dok na kartici Simulator prati njegovo izvršavanje. Za upravljanje izvršavanjem dostupne su komande Run, Step, Prev i Reset, a stanje se može pratiti u panelima za registre i memoriju. Listing paralelno prikazuje kolone Machine Code, Basic Code i Original Code, čime se omogućava povezivanje izvornog zapisa sa osnovnim instrukcijama i njihovim kodiranjem [5].

Dokumentacija navodi podršku za tačke prekida, vraćanje prethodnog koraka i određene pozive okruženja putem instrukcije ecall, među kojima su ispis i zahtev za dodatnim memorijskim prostorom [4]. Takvi pozivi pružaju dodatne mogućnosti programa unutar simulatora, ali ne treba ih poistovećivati sa simulacijom kompletnog operativnog sistema. Za potrebe ovog poređenja značajni su kao funkcionalnost koja nije uključena u razvijeno rešenje.

Venus je po načinu pristupa bliži aplikaciji iz ovog rada, jer se koristi kroz veb pregledač. Ipak, dostupnost gotove javne veb aplikacije treba razlikovati od lokalnog pokretanja razvijenog projekta, koje u dokumentovanom razvojnom postupku zahteva Node.js, instaliranje zavisnosti i Vite server. Sličnost platforme zato ne znači da su postupci pripreme i obim podržanih programa jednaki.

2.4. *Poređenje sa razvijenim simulatorom*

Jedna važna razlika odnosi se na to šta korisnik može da posmatra nakon asembliranja. RARS i Venus prikazuju i mašinski kod, pa se u njima može analizirati veza između tekstualne instrukcije i njenog kodiranja [2] [4]. Razvijeno rešenje zadržava se na internoj reprezentaciji i promeni stanja, zbog čega nije zamena za ove alate kada je cilj vežbe proučavanje bitskih polja instrukcija.

Druga razlika je način ponavljanja analize. Korak unazad u postojećim alatima omogućava povratak na prethodno stanje u okviru podržanih ograničenja [3] [4]. Razvijeni simulator ne čuva istoriju stanja: Reset vraća program na početak, a do željenog mesta ponovo se dolazi pojedinačnim koracima ili tačkom prekida. Ova razlika je važna kada korisnik želi da ponovi samo jednu instrukciju, umesto celog dela programa koji joj prethodi.

Razlike u podržanim instrukcijama i pozivima okruženja utiču i na prenosivost primera. Program koji u postojećem alatu koristi ispis preko ecall ili množenje ne može se bez prilagođavanja izvršiti u razvijenom simulatoru. Pri izboru primera zato je potrebno proveriti da li sve upotrebljene instrukcije i oblici zapisa pripadaju podržanom podskupu, a ne osloniti se samo na zajedničku oznaku RISC-V.

Sažetak odabranih mogućnosti prikazan je u Tabeli 1. Podaci za RARS i Venus preuzeti su iz navedenih izvora [2-5], dok kolona razvijenog simulatora opisuje implementaciju ovog rada. Oznaka da neka funkcija nije podržana odnosi se na trenutnu realizaciju, a ne na mogućnost njenog naknadnog dodavanja.

Tabela 1. Poređenje odabranih mogućnosti simulatora

Kriterijum	RARS	Venus	Razvijeni simulator
Okruženje	Java aplikacija	Veb aplikacija	Veb aplikacija
Skup instrukcija	32/64-bitni; više proširenja	RV32IM	15 instrukcija iz RV32I
Praćenje izvršavanja	Step, Go i tačke prekida	Step, Run i tačke prekida	Step, Run i tačke prekida
Registri i memorija	Prikaz stanja	Prikaz stanja	Prikaz upisivanih registara i reči
Mašinski kod	Prikazuje se	Prikazuje se	Ne generiše se
Korak unazad	Podrжан uz ograničenja	Podrжан	Nije podrжан
Pozivi okruženja	Podrжani	Podrжani	Nisu podrжani

Poređenje pokazuje da izvršavanje korak po korak, tačke prekida i prikaz stanja već postoje u posmatranim alatima. Doprinos ovog rada zato nije uvođenje tih mogućnosti kao novih, već njihova samostalna realizacija i povezivanje parsera, assemblera, modela procesora i korisničkog interfejsa. Izostavljanje mašinskog kodiranja i dodatnih instrukcija ograničava vrste programa i pojava koje se mogu prikazati.

Razvijeni simulator usmeren je na praćenje kraćih programa u okviru 15 instrukcija i 5 pseudo-instrukcija, uz izdvajanje registara i memorijskih reči u koje je upisivano. Takav obim omogućava da se u narednim poglavljima detaljno objasni ceo put od izvornog teksta do promene stanja. Jednostavniji prikaz je svesno izabran pri projektovanju. Međutim, bez ispitivanja sa korisnicima ne može se tvrditi da je razumljiviji ili da korisnicima više pomaže u učenju od prikaza u RARS-u i Venus-u.

3. Teorijske osnove RISC-V arhitekture i simulacije

Za razumevanje simulatora potrebno je povezati značenje instrukcija sa promenama registara, memorije i toka programa. U ovom poglavlju objašnjeni su osnovni pojmovi RISC-V arhitekture i uloge komponenti koje obrađuju i izvršavaju asemblerski kod. Pravila arhitekture navedena su odvojeno od ograničenja izabranih za potrebe ovog rada.

3.1. RISC-V i pojam arhitekture skupa instrukcija

Arhitektura skupa instrukcija (engl. Instruction Set Architecture, ISA) određuje koje operacije procesor pruža programu, kako se one zadaju i kako utiču na stanje vidljivo programeru. Istu arhitekturu mogu da realizuju procesori različite unutrašnje organizacije. Zato razumevanje značenja instrukcije ne zahteva poznavanje načina na koji je procesor hardverski realizovan.

RISC-V je porodica arhitektura u kojoj se osnovni skup instrukcija za rad sa celim brojevima dopunjuje proširenjima. Oznaka RV32I odnosi se na osnovni skup sa 32-bitnim celobrojnim registrima, dok se, na primer, množenje i deljenje dodaju proširenjem M [1]. Razvijeni simulator podržava samo odabranih 15 instrukcija iz RV32I; to ne predstavlja potpunu implementaciju tog skupa.

RV32I pripada arhitekturama tipa load-store: podaci se između memorije i registara prenose posebnim instrukcijama, dok aritmetičke operacije koriste vrednosti iz registara ili konstantu zadatu u instrukciji [6]. Za sabiranje vrednosti iz memorije zato je potrebno prvo učitati podatke, zatim izvršiti sabiranje i, po potrebi, upisati rezultat nazad. Ovakav sled jasno razdvaja računanje od pristupa memoriji.

3.2. Registri, programski brojač i memorija

RV32I ima 32 registra označena sa x0–x31, širine 32 bita. Registar x0 uvek sadrži nulu, a pokušaj upisa ne menja njegovu vrednost [6]. Ostali registri čuvaju operande, rezultate i adrese; u zapisima instrukcija oznaka rd predstavlja odredišni registar, a rs1 i rs2 izvorne registre.

Isti niz od 32 bita može se tumačiti kao broj sa znakom ili bez znaka. Za brojeve sa znakom koristi se dvojni komplement: zapis 0xFFFFFFFF, na primer, predstavlja -1, dok kao broj bez znaka predstavlja 4294967295 [1] [6]. Pri sabiranju i oduzimanju zadržavaju se donja 32 bita rezultata, pa prekoračenje opsega ne proširuje registar; značenje prikazane vrednosti zavisi od izabranog tumačenja.

Programski brojač, skraćeno PC (engl. Program Counter), čuva adresu instrukcije. Osnovne RV32I instrukcije imaju dužinu od 4 bajta, pa se pri redoslednom izvršavanju prelazi na adresu veću za 4 [6]. Grane i skokovi mogu da promene taj redosled; u simulatoru PC između dva koraka označava instrukciju koja će se sledeća izvršiti.

Memorija je bajtovski adresibilna: svaka adresa označava jedan bajt, a memorijska reč od 32 bita zauzima 4 uzastopna bajta. U simulatoru je izabran little-endian raspored, pri kome se najmanje značajan bajt čuva na najnižoj adresi. Na primer, reč 0x12345678 upisana od adrese 0 raspoređuje se kao 0x78, 0x56, 0x34 i 0x12 na adresama 0, 1, 2 i 3; specifikacija izbor redosleda bajtova prepušta izvršnom okruženju [6].

Razvijeno rešenje ima 4096 bajtova memorije podataka i zahteva da adresa za LW ili SW bude deljiva sa 4. To je pravilo ovog simulatora, dok ponašanje pri neporavnom pristupu u RISC-V zavisi od izvršnog okruženja [6]. Instrukcije su u aplikaciji sačuvane odvojeno od podataka, pa se LW i SW ne koriste za čitanje ili menjanje samog programa.

3.3. Grupe podržanih instrukcija i pseudo-instrukcije

Instrukcije ADD i SUB sabiraju i oduzimaju vrednosti iz dva registra, dok ADDI sabira vrednost registra sa konstantom navedenom u samoj instrukciji (engl. immediate). Operacije AND, OR i XOR deluju na odgovarajuće bitove operanada. SLL i SRL pomeraju bitove ulevo ili logički udesno, pri čemu se oslobođena mesta popunjavaju nulama, a broj mesta pomeranja određuje donjih 5 bita drugog izvornog registra [6].

Logičko pomeranje udesno treba razlikovati od aritmetičkog, koje pri pomeranju čuva znak broja. Ako registar sadrži -8, odnosno 0xFFFFFFF8, SRL za jedno mesto daje 0x7FFFFFFC, što je 2147483644, a ne -4. Razlika nastaje zato što SRL na najviše mesto postavlja nulu, umesto da ponovi prethodni znak; aritmetičko pomeranje udesno nije deo podržanog skupa ovog simulatora [6].

LW učitava memorijsku reč u registar, a SW upisuje reč iz registra u memoriju. Adresa pristupa dobija se sabiranjem vrednosti baznog registra i pomeraja navedenog u instrukciji [6]. Tako LW x3, 8(x2) učitava u x3 reč sa adrese koja je za 8 bajtova veća od vrednosti u x2.

BEQ i BNE menjaju tok programa kada su vrednosti dva registra jednake, odnosno različite. BLT vrši poređenje brojeva sa znakom i grana se kada je prvi operand manji od drugog [6]. Odredište grane određuje se pomerajem u odnosu na adresu same instrukcije grananja, a ne sledeće instrukcije; u izvornom kodu ovog simulatora odredište se zadaje labelom.

JAL izvršava skok sa pomerajem u odnosu na PC, dok JALR računa odredište iz baznog registra i pomeraja, uz postavljanje najnižeg bita odredišne adrese na nulu. Obe instrukcije u rd upisuju povratnu adresu, odnosno adresu instrukcije koja sledi posle skoka [6]. Izbor x0 za rd odbacuje tu adresu i omogućava skok bez njenog čuvanja.

Pseudo-instrukcija je pogodniji zapis koji assembler prevodi u jednu ili više stvarnih instrukcija; procesor je ne izvršava kao posebnu operaciju. U ovom simulatoru podržano je 5 pseudo-instrukcija i svaka se prevodi u jednu instrukciju, prema Tabeli 2. Oznaka imm predstavlja konstantu, rs izvorni registar, a labela simboličko ime odredišta skoka.

Tabela 2. Prevođenje pseudo-instrukcija u razvijenom simulatoru

Pseudo-instrukcija	Prevod u razvijenom simulatoru
LI rd, imm	ADDI rd, x0, imm
MV rd, rs	ADDI rd, rs, 0
NOP	ADDI x0, x0, 0
J labela	JAL x0, labela
RET	JALR x0, 0(x1)

Instrukcija ADDI koristi 12-bitnu konstantu sa znakom, čiji je opseg od -2048 do 2047 [6]. Pošto se LI u ovom rešenju uvek prevodi u ADDI, i za nju važi isti opseg; nije podržano učitavanje proizvoljne 32-bitne konstante jednom pseudo-instrukcijom. RET koristi x1 kao registar povratne adrese, u skladu sa uobičajenom konvencijom poziva funkcija [6].

3.4. Uloga parsera, assemblera i simulatora

Obrada asemblerskog programa i njegovo izvršavanje predstavljaju različite zadatke. Parser u ovom radu prepoznaje strukturu izvornog teksta: labele, nazive instrukcija i njihove operande, uz uklanjanje komentara i praznih linija. Rezultat tog koraka još ne potvrđuje da instrukcija postoji niti da se može izvršiti sa navedenim operandima.

Labela daje ime određenom mestu u programu, ali sama po sebi nije instrukcija. Zato broj linije u editoru nije isto što i adresa instrukcije: komentari, prazne linije i labele napisane u zasebnom redu ne zauzimaju prostor u nizu izvršivih instrukcija. Veza sa izvornom linijom ipak se čuva, kako bi korisnik mogao da prati izvršavanje i pronađe prijavljenu grešku.

Assembler proverava podržanost instrukcija, vrste i broj operanada, opsege konstanti i ispravnost labela. U prvom prolazu povezuje labele sa adresama, a u drugom prevodi instrukcije i računa pomeraje, što omogućava skok i na labelu definisanu kasnije u tekstu. Ako je, na primer, instrukcija grananja na adresi 12, a odredište na adresi 4, potreban pomeraj iznosi -8 bajtova.

U uobičajenom postupku asembliranja izlaz uključuje mašinski kod, ali se u ovom radu formira niz objekata koji opisuju instrukcije. Simulator zatim čita pripremljene operande i menja registre, memoriju i PC, bez ponovnog tumačenja izvornog teksta. Ovim se razdvaja ispravnost zapisa od ponašanja pri izvršavanju, dok korisnički interfejs omogućava praćenje dobijenog stanja.

Uspešno asembliranje ne znači da će se program izvršiti bez greške, jer dozvoljenost nekih operacija zavisi od izračunate adrese ili trenutnog stanja. Na primer, SW x1, 2(x0) ima ispravan oblik i pomeraj koji staje u dozvoljeni opseg, ali pokušava upis na adresu 2, koja u ovom simulatoru nije poravnata za pristup reči. Takva greška otkriva se pri izvršavanju, a ne pri proveru zapisa.

4. Korišćene tehnologije i razvojni alati

Izbor tehnologija zasnovan je na potrebi da simulator radi u veb pregledaču, da se njegova logika može proveravati nezavisno od interfejsa i da implementacija ostane razumljiva. TypeScript je korišćen za programsku logiku, HTML i CSS za strukturu i izgled stranice, a Vite i Vitest za razvoj, pripremu aplikacije i testiranje.

4.1. TypeScript

TypeScript dopunjuje JavaScript sistemom tipova koji omogućava otkrivanje određenih grešaka pre izvršavanja programa. Proverava se, na primer, da li funkcija dobija argument odgovarajućeg tipa i da li objekat sadrži polje koje kod pokušava da koristi [7]. U simulatoru je to važno jer više komponenti razmenjuje podatke o operandima, instrukcijama i greškama.

Tipovima su opisani mogući oblici operandada, struktura pripremljene instrukcije i rezultati obrade programa. Registar, konstanta i memorijski operand predstavljeni su različitim varijantama istog tipa, pa kod najpre proverava vrstu operandada, a zatim pristupa odgovarajućim podacima. Takav opis čini očekivani oblik podataka vidljivim u samom kodu, umesto da se programer oslanja samo na dogovor ili komentar.

U podešavanjima TypeScript-a uključena je opcija strict, koja omogućava strožu proveru tipova tokom razvoja aplikacije. Dodatno su uključene provere neiskorišćenih promenljivih i parametara, kao i provera da funkcije vraćaju vrednost u svim granama u kojima se ona očekuje. Strože provere obuhvataju i rukovanje vrednostima null i undefined [7], što je korisno, na primer, kada model procesora još ne postoji ili traženi element stranice nije pronađen. Ipak, tipovi sami ne proveravaju da li je asemblerska konstanta u dozvoljenom opsegu ili da li je memorijska adresa poravnata; takve provere moraju biti deo logike aplikacije.

4.2. HTML, CSS i neposredan rad sa DOM-om

HTML (engl. HyperText Markup Language) određuje strukturu stranice: editor, dugmad, izbor primera i panele za registre i memoriju. CSS (engl. Cascading Style Sheets) određuje njihov raspored, boje, razmake i način isticanja važnih elemenata. U ovom projektu raspored je realizovan kombinacijom mehanizama Grid i Flexbox, a posebne klase se koriste za tekuću instrukciju, tačke prekida i promenjene vrednosti.

Objektni model dokumenta, odnosno DOM (engl. Document Object Model), predstavlja elemente stranice kao objekte kojima program može da pristupa i menja ih [8]. Klik na dugme povezuje se sa odgovarajućom funkcijom, a posle izvršene akcije ažuriraju se listing, registri, memorija i status. Za dinamički ispis teksta koristi se svojstvo `textContent`, tako da se uneti asemblerski kod prikazuje kao tekst, a ne tumači kao HTML.

Nije korišćena dodatna biblioteka ili radni okvir za korisnički interfejs, poput React-a. Za postojeći broj panela i kontrola neposredan rad sa DOM-om omogućio je jednostavno povezivanje korisničkih akcija sa simulatorom, bez uvođenja dodatnih pojmova i zavisnosti. Pravila izvršavanja ipak nisu smeštena u kod za prikaz: interfejs poziva model procesora i prikazuje njegove rezultate, ali ne određuje značenje instrukcija.

4.3. Vite i razvojno okruženje

Vite obezbeđuje lokalni razvojni server i podršku za obradu modula i TypeScript datoteka [9]. Komanda `npm run dev` pokreće server preko kog se aplikacija otvara u pregledaču, uz osvežavanje prikaza nakon izmena koda. Time se izmene mogu proveravati tokom razvoja, bez ručne pripreme svih datoteka posle svakog uređivanja.

Važno je razlikovati prevođenje TypeScript-a u JavaScript od provere tipova: Vite obavlja prevođenje, ali sam ne sprovodi tu proveru [9]. U projektu komanda `npm run typecheck` zato posebno pokreće TypeScript proveru, dok `npm run build` najpre proverava tipove, a tek potom priprema datoteke aplikacije za isporuku. Opcija `noEmit` sprečava da taj korak provere pravi zasebne JavaScript datoteke, pošto je njihova priprema poverena Vite-u.

Node.js se koristi za pokretanje razvojnih alata, a npm za upravljanje zavisnostima i izvršavanje projektnih komandi. Vite priprema TypeScript kod same aplikacije za izvršavanje u pregledaču tako što ga prevodi u JavaScript. Kada korisnik unese RISC-V program u editor, već pokrenuta aplikacija u pregledaču pretvara taj tekst u niz instrukcija koje izvršava model procesora. Korisnički RISC-V program se, dakle, ne šalje Vite serveru radi asembliranja ili izvršavanja.

4.4. Vitest i provera tipova

Vitest je alat za automatizovano izvršavanje testova povezan sa Vite razvojnim okruženjem [10]. Odabran je zato što se uklapa u postojeće razvojno okruženje i omogućava direktno korišćenje TypeScript modula u testovima. Pošto parser, assembler i model procesora ne zavise od DOM-a, njihovi testovi ne zahtevaju otvaranje aplikacije u pregledaču.

Test priprema ulaz, poziva funkciju koja se proverava i poredi dobijeni rezultat sa očekivanim. Na primer, test sabiranja izvršava kratak program i proverava vrednost određiškog registra, dok test memorije proverava upisane bajtove. Komanda `npm test` pokreće jednu proveru svih testova, a `npm run test:watch` ponavlja odgovarajuće testove tokom izmena; rezultati i granični slučajevi detaljno su predstavljeni u sedmom poglavlju.

Provera tipova i testovi imaju različite uloge. Ispravno tipiziran kod može da sadrži pogrešno računanje adrese ili rezultata instrukcije, što se otkriva testom ponašanja, dok provera tipova može da uoči pogrešnu upotrebu podataka pre pokretanja testa. Njihova kombinacija omogućava proveru različitih vrsta grešaka, ali ne zamenjuje ručnu proveru prikaza i rada korisničkih kontrola.

Reference

- [1] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, izdanje 20260120, Introduction: RISC-V ISA Overview. <https://docs.riscv.org/reference/isa/v20260120/unpriv/intro.html> [pristupljeno 28.09.2026.].
- [2] TheThirdOne/rars. RARS — RISC-V Assembler and Runtime Simulator, opis projekta. <https://github.com/TheThirdOne/rars> [pristupljeno 28.09.2026.].
- [3] TheThirdOne/rars. Interactive Debugging Features, dokumentacija projekta. <https://github.com/TheThirdOne/rars/blob/master/src/help/Debugging.html> [pristupljeno 28.09.2026.].
- [4] kvakil/venus. Venus, opis projekta. <https://github.com/kvakil/venus> [pristupljeno 28.09.2026.].
- [5] Venus. Veb interfejs simulatora. <https://venus.kvakil.me/> [pristupljeno 28.09.2026.].
- [6] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, izdanje 20260120, RV32I Base Integer Instruction Set, Version 2.1. <https://docs.riscv.org/reference/isa/v20260120/unpriv/rv32.html> [pristupljeno 28.09.2026.].
- [7] TypeScript. The TypeScript Handbook: The Basics. <https://www.typescriptlang.org/docs/handbook/2/basic-types.html> [pristupljeno 28.09.2026.].
- [8] MDN Web Docs. Document Object Model (DOM). https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model [pristupljeno 28.09.2026.].
- [9] Vite. Features. <https://vite.dev/guide/features.html> [pristupljeno 28.09.2026.].
- [10] Vitest. Getting Started. <https://vitest.dev/guide/> [pristupljeno 28.09.2026.].